Phase 1 — Prepare the files

Step 1. Open the lab computer

1. Log in to Windows.
2. Open Xilinx ISE Design Suite 14.5 → Project Navigator.
3. Wait for the main window to open.

Your earlier screenshot showed a project called New_Project containing Half_Adder.v. That is a different example project.

Do not add your decoder files to the Half Adder project. Keep it untouched.

Step 2. Locate your existing VHDL files

Before creating the project, make sure the lab computer can access your complete, current set of VHDL source files.

Your project includes files such as:

- tb_full_i8.vhd
- tx_chain_i8.vhd
- rx_chain_i8.vhd
- RS encoder and decoder source files
- Interleaver and deinterleaver source files
- Viterbi decoder and frame-wrapper source files
- Any supporting packages, components, and required design files

The exact file list depends on the code you currently have. Include every file required by the hierarchy.

Do not recreate the VHDL code from memory. Copy your existing files using a lab-approved method, such as an authorized USB drive, if permitted. If the files aren't available on the lab computer, you must obtain them before testing.

Phase 2 — Create a new ISE project

Step 3. Start the New Project wizard

1. In Project Navigator, click File in the top menu.
2. Click New Project.
3. A wizard window should appear.

Step 4. Enter the project details

Fill in the fields as follows.

Project Name

RS_CC_Decoder_Sim

Copy

Project Location

Choose a folder you can access and write to, for example your Documents folder.

Top-Level Source Type

HDL

Click Next.

Step 5. Configure the project

ISE may ask for device and design settings even though you only want simulation.

- Family / Device / Package / Speed: choose valid settings available in the lab installation. If your instructor has specified a target or a lab template, follow that. You don't need to program this device.
- Preferred Language: choose VHDL.
- Simulator: choose ISim (VHDL/Verilog) if this option appears.

Click Next through the wizard, review the summary, and click Finish.

If your installation displays different options, don't guess at an unsupported device or simulator. Use the choices actually available in that installation.

Phase 3 — Add your VHDL design files

Step 6. Add the design source files

1. In the top menu, click Project → Add Source.
2. Navigate to the folder containing your existing VHDL files.
3. Select the RS, CC, interleaver, deinterleaver, transmitter, receiver, Viterbi, wrapper, and other required design files.
4. Click Open.
5. If ISE asks how to associate the files, add the design modules as design sources.
6. Confirm the files appear in the Sources pane.

You may select multiple files at once if Windows allows it. If you're unsure whether a file is required, include it rather than deleting or rewriting anything.

Step 7. Add the testbench

This step is separate from adding the design files.

1. Click Project → Add Source again.
2. Navigate to tb_full_i8.vhd.
3. Select it and click Open.
4. If an association dialog appears, choose Simulation if offered.
5. Confirm the file has been added to the project.

If ISE adds the file under a different association, you can still change the source view and check whether ISE recognizes it as a simulation source.

Step 8. Switch the Sources pane to Simulation

At the upper-left of Project Navigator, your screenshot showed two views: Implementation and Simulation.

1. Click Simulation.
2. Look for tb_full_i8 in the hierarchy.
3. Expand the hierarchy if necessary to see its dependencies.

You should be able to distinguish the testbench from its instantiated design modules.

Remember: tb_full_i8 is the full-chain testbench. Don't choose the standalone RS testbench or Viterbi testbench for this particular test.

Phase 4 — Select the correct simulation top

Step 9. Set tb_full_i8 as the top module

1. In the Simulation view, click tb_full_i8.
2. Right-click the file or module name.
3. If Set as Top Module appears, click it.
4. Confirm the hierarchy shows tb_full_i8 as the top of the simulation.

If tb_full_i8 doesn't appear, check that you added the file and that its entity is actually named tb_full_i8.

If ISE reports that an entity cannot be found, don't start editing code yet. First check that all source files are included and that the correct source view is selected.

Phase 5 — Check compilation before running

Step 10. Locate the simulation processes

With tb_full_i8 selected, look at the Processes pane, usually below the Sources pane.

Expand the section called ISim Simulator if it appears.

Look for processes such as:

- Behavioral Check Syntax
- Simulate Behavioral Model

The wording and availability can vary with the installed ISE configuration.

Step 11. Run Behavioral Check Syntax

1. Double-click Behavioral Check Syntax.
2. Wait for the process to finish.
3. Check the Console or Errors pane at the bottom.

If it passes, continue to the simulation.

If it fails, note the first actual compiler error, including the file and line number. Later errors may simply be consequences of the first one.

Common categories of compatibility errors include:

- Unsupported VHDL language features.
- Missing source files or packages.
- Entity or component name mismatches.
- Testbench constructs not supported by the installed simulator.

Do not change the working code just because the project displays warnings.

Phase 6 — Run the complete simulation

Step 12. Start ISim

1. Select tb_full_i8 in the Simulation hierarchy.
2. In the Processes pane, double-click Simulate Behavioral Model.
3. Wait for ISim to compile the design and open the simulator.
4. If a waveform window opens, allow the simulation to initialize.

If ISim fails during compilation, return to the first error message rather than repeatedly trying to run the simulation.

Step 13. Run the simulation

Once ISim opens successfully:

1. Find the simulation run controls.
2. Run the simulation for a bounded amount of time, or use the testbench's configured run duration if one is provided.
3. Check the transcript for the testbench's PASS message.

Important: Your QuestaSim testbench previously reported the full-chain PASS at around 184206 ns. That is a useful reference, but it does not guarantee ISim will finish at exactly the same time.

Avoid running indefinitely if the testbench contains a clock that continues forever. If ISim has a Run All button, use it only if the testbench has a reliable stopping condition.

Phase 7 — Verify that the test passed

Step 14. Check the transcript

Your testbench should report a message equivalent to:

PASS: depth-8 receiver recovered original data after single-bit error

That is the key result from your existing full-chain testbench.

Confirm that:

- The testbench reaches its PASS condition.
- The original input data matches the recovered receiver data.
- There is no earlier compilation or runtime failure that invalidates the result.

If ISim shows a different message, inspect the transcript carefully. A message such as PASS should come from the actual testbench; don't assume the simulation passed merely because the simulator opened.

Step 15. Check the waveform if necessary

If you need to demonstrate the simulation visually:

1. Add the main testbench signals to the waveform window.
2. Include clk, reset, start, tx_done, rx_done, rx_error, and the receiver output.
3. Run the simulation to the relevant point.
4. Use the waveform zoom controls to inspect the completion interval.

You do not need to add every internal signal. The full-chain PASS message and matching input/output data are the important checks.

Phase 8 — What to do if an error appears

Error or situation	What to check first
tb_full_i8 is missing	Confirm that the testbench was added and the view is set to Simulation.
Entity not found	Confirm all required source files are included.
Syntax error	Record the first error, file, and line number.
Missing package or component	Check that its source file is included and compiled before its dependent module.

Simulation opens but does not finish	Check the transcript and testbench stopping condition.
Simulation runs but no PASS appears	Check the testbench's assertions and the final transcript.
QuestaSim passes but ISim fails	Investigate the specific ISim error before changing any RTL.

1. Initial condition and reset

At the beginning, the clock starts toggling continuously. The reset signal is asserted initially, so the transmitter and receiver are placed in their initial states. The transmitter state is IDLE, and the receiver is also in IDLE.

At this stage, start is low, so the transmitter has not received a request to begin processing. Consequently, tx_done remains low because transmission has not yet completed.

2. Start signal becomes high

When start becomes high, the transmitter receives the command to begin processing the input data. The transmitter starts the RS encoding operation.

The tx_done signal is still low because the transmitter has only started working; it has not completed the entire transmission process.

3. Transmitter enters START_RS and WAIT_RS

Now observe the transmitter's state signal. It changes from IDLE to START_RS, and then to WAIT_RS.

In START_RS, the transmitter initiates the RS encoder operation. After starting it, the transmitter enters WAIT_RS and waits for the RS encoder to finish processing the current codeword.

Once the encoder completes its operation, the controller proceeds to the next codeword and repeats the process. This is why you can see the sequence START_RS, WAIT_RS, START_RS, WAIT_RS repeating in the waveform.

4. Symbol index and codeword selection change

Now look at symbol_index, code_index, and codeword_select.

Initially, these signals are zero because the transmitter is at the beginning of the encoding process. As processing advances, the relevant index signals increment or change to select the next symbol or codeword.

For example, code_index progresses through 0, 1, 2, 3, and 4 in the visible waveform. This indicates that the controller is progressing through successive codewords.

The important point is that the transmitter is processing the data in stages rather than transmitting the entire frame in a single clock cycle.

5. Interleaver output begins to change

Initially, the interleaved signal contains zeros because the interleaver output has not yet been populated with the encoded data.

As the RS encoder generates codewords and the interleaving process advances, the interleaver output begins to change. The bit_index tracks the position being processed.

The interleaver reorganizes the RS-coded data according to the configured interleaving depth of eight. This helps distribute errors across codewords so that the RS decoder can correct them more effectively.

6. Convolutional encoder processes the interleaved data

Next, the interleaved data is supplied to the convolutional encoder.

The cc_data signal carries the input bit, while cc_valid indicates when that input bit is valid. The encoder generates two output bits for each input bit because the convolutional code has a rate of 1/2.

The cc_code_out signal carries these encoded bits, and cc_valid_out indicates valid encoded output. The transmitter collects the encoded bits into tx_reg.

The encoded frame contains 32,640 bits, twice the 16,320-bit input to the convolutional encoder.

7. Transmission completes

Once all the required data has passed through the transmitter processing stages, the transmitter finishes its work and asserts tx_done.

Before this point, tx_done remains low because the complete transmission has not finished. The important distinction is that starting transmission and completing transmission are two different events.

8. Receiver begins processing

After the encoded frame is available, the receiver begins its processing sequence. Its control signals indicate when reception and decoding start, whether the receiver is busy, and when processing finishes.

The received encoded bits are stored or latched for processing by the Viterbi decoder.

9. Viterbi decoder recovers the convolutionally encoded data

The Viterbi decoder processes the received bits and determines the most likely original bit sequence. The vit_start, vit_busy, and vit_done signals indicate the decoder's progress.

The decoded_bits signal contains the output recovered by the Viterbi decoder. This output is then passed to the deinterleaver and RS-decoding stages.

10. Deinterleaver restores the RS codeword order

The deinterleaver reverses the interleaving arrangement used by the transmitter. The deinterleaved_bits signal carries the data after this reordering.

The purpose of this stage is to restore the data to the arrangement expected by the RS decoder.

11. RS decoder corrects the symbol error

The RS decoder processes the restored codewords. The rs_valid, rs_error, and rs_done signals indicate the relevant decoding status and completion.

In this test, a single-bit error is deliberately introduced into the transmitted frame. After convolutional decoding, deinterleaving, and RS decoding, the receiver is expected to recover the original data.

12. Final output matches the original input

Finally, the receiver's rx_data is compared against original_data.

The full-chain testbench reports:

PASS: depth-8 receiver recovered original data after single-bit error

The final PASS occurs at approximately 184,206 ns. This confirms that the simulated receiver recovered the original data despite the injected error.

Overall sequence: Reset → Start → RS encoding → Interleaving → Convolutional encoding → Transmission → Viterbi decoding → Deinterleaving → RS decoding → Recovered output.

One important detail: your screenshots show the waveform zoomed into approximately the first 190 ns, whereas the full simulation finishes at 184,206 ns. Therefore, the early screenshot shows the initial control transitions and the start of processing; the final recovery must be inspected near the end of the simulation.

Full waveform explanation — RS + CC decoder

1. Initialisation and reset

- Initially, clk toggles continuously and reset initializes the transmitter and receiver.
- Both state machines begin in IDLE, and start is initially 0.
- tx_done and rx_done remain 0 because transmission and reception have not completed.

2. Transmitter starts RS encoding

- When start becomes 1, the transmitter begins processing the original data.
- The transmitter state changes to START_RS, and rs_start initiates RS encoding.
- It then enters WAIT_RS to wait for the encoder to finish processing the current codeword.
- This process repeats for successive codewords.

3. Codeword and symbol selection

- code_index advances through the codewords, while symbol_index tracks the symbol position.
- codeword_select selects the codeword being processed by the interleaver.
- These signals change as the transmitter progresses through the input data.

4. Interleaver

- Initially, interleaved contains zeros because the output has not yet been populated.
- As RS-coded data becomes available, the interleaver output begins to show non-zero hexadecimal values.
- bit_index tracks the data position, and the interleaver rearranges the data according to depth 8.

5. Transmitter preparation

- After the RS encoding sequence, the transmitter enters PREPARE.
- It prepares the interleaved data for convolutional encoding and transmission.
- tx_done remains 0 because the complete transmission has not finished.

6. Convolutional encoder

- In SEND_BITS, cc_data supplies input bits to the convolutional encoder.
- cc_valid = 1 indicates that the input bit is valid.
- For each input bit, cc_code_out produces two output bits, such as 01, 10, 00, or 11.
- cc_valid_out indicates valid encoded output.
- cc_count tracks the encoding progress, while tx_reg accumulates the encoded bits.
- Because the code rate is 1/2, the output length is twice the input length.

7. Transmission completes

- The transmitter state progresses through SEND_BITS, WAIT_CC, COMPLETE, and IDLE.
- tx_busy becomes 0 when the transmitter finishes.
- tx_done indicates that the transmission process has completed.

8. Receiver captures the frame

- rx_start initiates receiver processing.

- rx_busy indicates that the receiver is active.
- rx_latched stores the received encoded frame so that the decoder can process it.
- The receiver passes the encoded bits to the Viterbi decoder.

9. Viterbi decoding

- vit_start starts Viterbi decoding.
- vit_busy indicates that the decoder is processing the received bits.
- The Viterbi state progresses through its decoding and result stages before returning to IDLE.
- core_decoded contains the decoded frame, and decoded_bits carries the recovered data to the next stage.
- Viterbi decoding uses the redundancy added by convolutional encoding to recover the most likely original bit sequence.

10. Deinterleaver

- deinterleaved_bits restores the RS-coded data to its original codeword order.
- This reverses the interleaving arrangement applied at the transmitter.
- The restored data is then supplied to the RS decoder.

11. RS decoding and error correction

- rs_start initiates RS decoding, and rs_busy indicates that decoding is in progress.
- rs_valid indicates valid decoder data.
- rs_error indicates the RS error status.
- rs_done indicates completion of RS decoding.
- The RS decoder corrects the injected error and recovers the original information data.

12. Final data recovery

- original_data contains the original input frame, while rx_data contains the recovered frame.
- In your waveform, both buses begin with 03 0A 11 18, showing that the displayed initial bytes match.
- The full-chain testbench reports PASS at 184,206 ns, confirming that the receiver recovered the original data after the injected single-bit error.

Overall flow: Reset → RS encoding → Interleaving → Convolutional encoding → Transmission → Viterbi decoding → Deinterleaving → RS decoding → Original data recovery.